

# Memory Management

Liting Zhang

University of West Florida

# Outline

- Overview: what memory management means in C++
- Memory regions: code, static, stack, heap
- Stack: automatic allocation, function calls, stack overflow
- Heap: dynamic allocation with new and delete
- Pointer basics: address-of, dereference, nullptr
- Dynamic arrays: 1D and 2D patterns
- References vs pointers
- Classes owning dynamic memory: destructor, shallow copy, rule of 3
- Rule of 5 and move semantics
- Tools: valgrind (Linux)

# Learning goals

After this lecture, you should be able to

- Describe the major memory regions and what lives in each one
- Explain stack vs heap allocation and common failure modes
- Use pointers safely: initialization, dereference rules, nullptr
- Allocate and free single objects and arrays correctly
- Build dynamic arrays (1D) and 2D arrays on the heap
- Explain memory bugs: leaks, double delete, use-after-free, out-of-bounds
- Explain shallow copy and why it breaks resource-owning classes
- Implement rule of 3 and describe rule of 5 at a high level

# Overview

Definition: memory management in C++ is allocating and deallocating memory for objects and data structures during program execution.

Motivation:

- Optimize memory usage
- Avoid memory leaks
- Avoid dangling pointers
- Avoid stack overflow

# Memory Regions

- 1 Memory Regions
- 2 Stack
- 3 Heap
- 4 Pointers
- 5 Dynamic 1D Arrays
- 6 2D Arrays
- 7 References vs Pointers
- 8 Classes Owning Dynamic Memory
- 9 Rule of 3
- 10 Rule of 5
- 11 Tools
- 12 Review

# Four regions

- Code: compiled program instructions
- Static: global and static variables
- Stack: local variables and function parameters
- Heap: dynamically allocated memory

Each region has different lifetime and management rules.

# Memory addresses

In RAM, memory is a big sequence of numbered slots.

- Each slot has an address (like house numbers)
- Lowest memory address is a small number
- Highest memory address is a large number
- A pointer stores an address

# Typical layout intuition

Common mental model:

- Stack and heap share a free region between them
- Stack grows as functions call functions
- Heap grows as you allocate dynamic memory
- If they collide: crash (stack overflow or out of memory)



## Memory regions at a glance

| Region | Typical contents                                            | Lifetime               | Management / notes                                                                                                              |
|--------|-------------------------------------------------------------|------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Code   | Program instructions, compiled functions                    | Entire program run     | Read-only in many systems; not for variable storage                                                                             |
| Static | Global variables, static variables, string literals (often) | Entire program run     | Allocated once; useful for shared state, but can increase coupling                                                              |
| Stack  | Function parameters, local variables, return addresses      | Function call duration | Automatic allocation and cleanup; limited size; deep recursion can overflow                                                     |
| Heap   | Dynamically allocated objects and arrays                    | Until manually freed   | Allocated with <code>new/new[]</code> and freed with <code>delete/delete[]</code> ; leaks and dangling pointers are common bugs |

Layout varies by OS and compiler, but these roles are consistent.

# Typical address space picture

|                  |                                                                    |
|------------------|--------------------------------------------------------------------|
| Higher addresses | Stack region (often grows downward as calls happen)                |
|                  | Free space between stack and heap (shrinks as stack and heap grow) |
|                  | Heap region (often grows upward as new allocations happen)         |
|                  | Static region (globals and static variables)                       |
| Lower addresses  | Code region (program instructions)                                 |

Use this picture to explain collisions: stack overflow, out-of-memory, or stack/heap meeting.

# Stack

- 1 Memory Regions
- 2 Stack**
- 3 Heap
- 4 Pointers
- 5 Dynamic 1D Arrays
- 6 2D Arrays
- 7 References vs Pointers
- 8 Classes Owning Dynamic Memory
- 9 Rule of 3
- 10 Rule of 5
- 11 Tools
- 12 Review

# Stack memory management

- Stack is LIFO: last in, first out
- The runtime maintains a call stack
- Stores local variables and function parameters
- Automatically managed: allocation and deallocation happen for you

Stack overflow happens when the stack cannot grow further.

# Stack overflow: common causes

- Infinite recursion (no base case)
- Huge local arrays on the stack
- Deep recursion with large stack frames

Recursion details are discussed in the recursion lecture, but this is the key failure mode.

# Practice 1: identify stack overflow risk

Predict what can go wrong.

```
1 | int f(int n) {  
2 |     int big[1000000];  
3 |     if (n <= 0) return 0;  
4 |     return big[0] + f(n - 1);  
5 | }  
6 | int main() {  
7 |     return f(10);  
8 | }
```

- What is the risk
- What is one safer redesign

# Practice 1 Answer

- Risk: big is allocated on the stack for every recursive call. This can overflow quickly.
- Safer redesign:
  - ▶ move large memory to the heap once, reuse it
  - ▶ or redesign to iterative
  - ▶ or reduce stack frame size

# Heap

- 1 Memory Regions
- 2 Stack
- 3 Heap**
- 4 Pointers
- 5 Dynamic 1D Arrays
- 6 2D Arrays
- 7 References vs Pointers
- 8 Classes Owning Dynamic Memory
- 9 Rule of 3
- 10 Rule of 5
- 11 Tools
- 12 Review



# Heap memory management

- Heap is used for dynamic memory
- Managed manually by the programmer
- Allocate with new
- Deallocate with delete
- Dynamic objects are accessed through pointers

# Dynamic data facts and rules

- A pointer is not the object: it stores the address of the object
- The dynamic object lives on the heap until you destroy it
- delete on nullptr does nothing
- Match form to form:
  - ▶ new with delete
  - ▶ new[] with delete[]
- Each allocation must be freed exactly once

# Typical bugs to avoid

- Memory leak: forget to delete
- Double delete: delete same block twice
- Use after free: use pointer after deleting it
- Out of bounds access: write past allocated size
- Invalid pointer casts: reinterpret addresses incorrectly

# Single object vs array allocation

```
1 // single object
2 MyClass* p = new MyClass();
3 delete p;
4
5 // array
6 int* a = new int[10];
7 delete[] a;
```

## Practice 2: leak or not

Identify the bug.

```
1  int* makeArray(int n) {  
2      int* a = new int[n];  
3      for (int i = 0; i < n; i++) a[i] = i;  
4      return a;  
5  }  
6  int main() {  
7      int* p = makeArray(100);  
8      // use p  
9      return 0;  
10 }
```

- What is wrong
- Show one fix

## Practice 2 Answer

What is wrong:

- `p` points to dynamic memory that is never freed.

One fix:

```
1 | int main() {  
2 |     int* p = makeArray(100);  
3 |     // use p  
4 |     delete[] p;  
5 |     return 0;  
6 | }
```

# Pointers

- 1 Memory Regions
- 2 Stack
- 3 Heap
- 4 Pointers**
- 5 Dynamic 1D Arrays
- 6 2D Arrays
- 7 References vs Pointers
- 8 Classes Owning Dynamic Memory
- 9 Rule of 3
- 10 Rule of 5
- 11 Tools
- 12 Review

# Pointer definition

A pointer is a variable that stores an address of an object of some type.

- The pointee is the object that lives at that address
- The pointer variable itself also has an address
- Use cases:
  - ▶ dynamic-size data
  - ▶ share data without copying
  - ▶ build linked structures



# Pointer basic syntax

```
1 | int x = 22;  
2 | int* ptr = &x;    // ptr stores address of x  
3 | *ptr = 30;        // dereference: modify x through ptr
```

Rule: never dereference a pointer until you know it is valid.

# Null pointer

Initialize pointers to `nullptr` to mean no object.

- Checking before dereference can prevent crashes
- `delete` and `delete[]` on `nullptr` does nothing

## Practice 3: nullptr crash

What happens, and how do you fix it?

```
1 | int main() {  
2 |     int* p = nullptr;  
3 |     *p = 5;  
4 | }
```

## Practice 3 Answer

- Dereferencing nullptr crashes (undefined behavior).
- Fix: point to a real object before dereference.

```
1 | int main() {  
2 |     int x = 0;  
3 |     int* p = &x;  
4 |     *p = 5;  
5 | }
```

# Dynamic 1D Arrays

- 1 Memory Regions
- 2 Stack
- 3 Heap
- 4 Pointers
- 5 Dynamic 1D Arrays**
- 6 2D Arrays
- 7 References vs Pointers
- 8 Classes Owning Dynamic Memory
- 9 Rule of 3
- 10 Rule of 5
- 11 Tools
- 12 Review

# Dynamic 1D array idea

Static arrays have compile-time fixed size.

- Example: `int arr[10];`
- Sometimes size is only known at runtime
- Dynamic arrays allocate memory on the heap

# Dynamic 1D array example

```
1  int n = 0;
2  cin >> n;
3  int* arr = new int[n];
4
5  for (int i = 0; i < n; i++) arr[i] = i * i;
6
7  delete[] arr;
8  arr = nullptr;
```

## Example class for dynamic arrays

```
1 class State {  
2 private:  
3     string name;  
4 public:  
5     State() : name("") {}  
6     State(const string& n) : name(n) {}  
7     string getName() const { return name; }  
8 };
```



# Two approaches for dynamic arrays of objects

- Contiguous block: one allocation for  $N$  objects
- Pointer table: one allocation for  $N$  pointers, then  $N$  allocations for objects

Tradeoff:

- Contiguous is cache-friendly and easier to delete
- Pointer table supports per-element lifetime flexibility, but is more complex

# Dynamic 1D objects: contiguous block

```
1  size_t N = 3;
2  State* states1 = new State[N];
3
4  for (size_t i = 0; i < N; i++) {
5      states1[i] = State("Alabama");
6  }
7
8  cout << states1[0].getName() << "\n";
9
10 delete[] states1;
11 states1 = nullptr;
```

# Dynamic 1D objects: pointer table

```
1  size_t N = 3;
2  State** states2 = new State*[N]();
3
4  for (size_t i = 0; i < N; i++) {
5      states2[i] = new State("Alabama");
6  }
7
8  cout << states2[0]->getName() << "\n";
9
10 for (size_t i = 0; i < N; i++) {
11     delete states2[i];
12 }
13 delete[] states2;
14 states2 = nullptr;
```

# 2D Arrays

- 1 Memory Regions
- 2 Stack
- 3 Heap
- 4 Pointers
- 5 Dynamic 1D Arrays
- 6 2D Arrays**
- 7 References vs Pointers
- 8 Classes Owning Dynamic Memory
- 9 Rule of 3
- 10 Rule of 5
- 11 Tools
- 12 Review

## 2D arrays on the stack

- Fixed-size 2D array is contiguous in row-major order
- Column count must be compile-time constant for typical function parameters
- Works well when dimensions are known at compile time

## 2D array on stack example

```
1  const int ROWS = 2;
2  const int COLS = 5;
3
4  double mat[ROWS][COLS] = {
5      {1, 2, 3, 4, 5},
6      {6, 7, 8, 9, 10}
7  };
8
9  cout << mat[1][2] << "\n";
```

## 2D arrays on heap: two common patterns

- One contiguous block: allocate  $\text{rows} * \text{cols}$  as one 1D array
- Pointer-to-pointer: allocate an array of row pointers, then allocate each row

Key difference:

- One block is contiguous overall
- Pointer-to-pointer is not contiguous overall

# Heap 2D as one contiguous block

Manual indexing:  $\text{index} = r * \text{COLS} + c$

```
1  size_t ROWS = 2;
2  size_t COLS = 5;
3
4  double* block = new double[ROWS * COLS];
5
6  for (size_t r = 0; r < ROWS; r++) {
7      for (size_t c = 0; c < COLS; c++) {
8          block[r * COLS + c] = r * COLS + c + 1;
9      }
10 }
11
12 cout << block[1 * COLS + 2] << "\n";
13
14 delete[] block;
15 block = nullptr;
```



# Heap 2D via pointer-to-pointer

```
1  size_t ROWS = 2;
2  size_t COLS = 5;
3
4  double** dyn = new double*[ROWS];
5
6  for (size_t r = 0; r < ROWS; r++) {
7      dyn[r] = new double[COLS];
8      for (size_t c = 0; c < COLS; c++) {
9          dyn[r][c] = r * COLS + c + 1;
10     }
11 }
12
13 cout << dyn[1][2] << "\n";
14
15 for (size_t r = 0; r < ROWS; r++) {
16     delete[] dyn[r];
17 }
18 delete[] dyn;
19 dyn = nullptr;
```

## Practice 4: missing cleanup

What is missing?

```
1 double** make2D(size_t rows, size_t cols) {  
2     double** a = new double*[rows];  
3     for (size_t r = 0; r < rows; r++) {  
4         a[r] = new double[cols];  
5     }  
6     return a;  
7 }  
8 int main() {  
9     double** m = make2D(3, 4);  
10    // use m  
11    delete[] m;  
12 }
```

- What bug occurs
- How do you fix the cleanup

## Practice 4 Answer

- Bug: each row allocation leaks because only the outer pointer array is freed.
- Fix: delete each row first, then delete the pointer table.

```
1 int main() {  
2     double** m = make2D(3, 4);  
3  
4     for (size_t r = 0; r < 3; r++) {  
5         delete[] m[r];  
6     }  
7     delete[] m;  
8     m = nullptr;  
9 }
```

# References vs Pointers

- 1 Memory Regions
- 2 Stack
- 3 Heap
- 4 Pointers
- 5 Dynamic 1D Arrays
- 6 2D Arrays
- 7 References vs Pointers**
- 8 Classes Owning Dynamic Memory
- 9 Rule of 3
- 10 Rule of 5
- 11 Tools
- 12 Review

# Reference vs pointer

## Reference

- Alias for another variable
- Must bind to something at creation
- Cannot be nullptr
- Cannot be reseated later

## Pointer

- Stores an address
- Can be nullptr
- Can be reseated to point elsewhere
- Requires dereference to access the pointed value

# Example: swap with pointers vs references

Pointer version:

```
1 void swapPtr(int* p1, int* p2) {  
2     int tmp = *p1;  
3     *p1 = *p2;  
4     *p2 = tmp;  
5 }
```

Reference version:

```
1 void swapRef(int& a, int& b) {  
2     int tmp = a;  
3     a = b;  
4     b = tmp;  
5 }
```

## Practice 5: choose pointer or reference

You are writing a function that may or may not modify an integer.

- If the function must always receive a valid integer, what is a good choice
- If the function should allow no integer (no object), what is a good choice

## Practice 5 Answer

- Always valid integer: reference is a good choice
- Allow no integer: pointer is a good choice, because nullptr can represent no object



# Classes Owning Dynamic Memory

- 1 Memory Regions
- 2 Stack
- 3 Heap
- 4 Pointers
- 5 Dynamic 1D Arrays
- 6 2D Arrays
- 7 References vs Pointers
- 8 Classes Owning Dynamic Memory**
- 9 Rule of 3
- 10 Rule of 5
- 11 Tools
- 12 Review

# Typical fields for a dynamic array class

For a vector-like class using a dynamic array:

- pointer field: address of dynamic array
- size field: number of elements currently stored
- capacity field: total elements that can fit before growing

# Growth strategy

- Expanding by  $+1$  each time is expensive
- Common approach: double capacity when full
- This gives amortized constant-time `push_back` behavior

# Destructor idea

A destructor runs automatically when an object is destroyed.

- Resource-owning classes should release dynamic memory in the destructor
- Arrays allocated with `new[]` must be destroyed with `delete[]`

# Destructor example

```
1 class List {
2     int size;
3     int* arr;
4 public:
5     List() : size(0), arr(nullptr) {}
6
7     List(int count, int value = 0) {
8         size = count;
9         arr = new int[count];
10        for (int i = 0; i < count; i++) arr[i] = value;
11    }
12
13    ~List() {
14        delete[] arr;
15        arr = nullptr;
16    }
17};
```

# Shallow copy problem

Default copy behavior for a class with a pointer:

- Copy constructor copies pointer value only
- Copy assignment copies pointer value only
- Two objects now share one heap block
- Destructor runs twice on same block

Result: double delete crash and shared-state surprises.

## Practice 6: predict the crash

What goes wrong?

```
1  class Bag {
2      int n;
3      int* data;
4  public:
5      Bag(int n_) : n(n_) { data = new int[n]; }
6      ~Bag() { delete[] data; }
7  };
8
9  int main() {
10     Bag a(5);
11     Bag b = a;
12 }
```

- What is the bug
- What rule do we need

## Practice 6 Answer

- Bug: default copy makes a shallow copy. a and b share the same data pointer.
- When both destructors run, `delete[]` happens twice.
- Fix: rule of 3 (deep copy) for resource-owning classes.



# Rule of 3

- 1 Memory Regions
- 2 Stack
- 3 Heap
- 4 Pointers
- 5 Dynamic 1D Arrays
- 6 2D Arrays
- 7 References vs Pointers
- 8 Classes Owning Dynamic Memory
- 9 Rule of 3**
- 10 Rule of 5
- 11 Tools
- 12 Review

# Rule of 3

If a class owns a resource (dynamic memory), you typically need:

- Destructor
- Copy constructor
- Copy assignment operator

Goal: deep copy so each object owns its own heap block.

## Deep copy: copy constructor

```
1  class List {
2      int size;
3      int* arr;
4  public:
5      List() : size(0), arr(nullptr) {}
6      List(int count, int value = 0) {
7          size = count;
8          arr = new int[count];
9          for (int i = 0; i < count; i++) arr[i] = value;
10     }
11     ~List() { delete[] arr; }
12     List(const List& other) {
13         size = other.size;
14         arr = new int[size];
15         for (int i = 0; i < size; i++) arr[i] = other.arr[i];
16     }
17 };
```

# Deep copy: copy assignment operator

Must handle self-assignment and existing memory.

```
1  class List {
2      int size;
3      int* arr;
4  public:
5      List() : size(0), arr(nullptr) {}
6      List(int count, int value = 0) {
7          size = count;
8          arr = new int[count];
9          for (int i = 0; i < count; i++) arr[i] = value;
10     }
11     ~List() { delete[] arr; }
12     List(const List& other) {
13         size = other.size;
14         arr = new int[size];
15         for (int i = 0; i < size; i++) arr[i] = other.arr[i];
16     }
17     List& operator=(const List& other) {
18         if (this == &other) return *this;
19         delete[] arr;
20         size = other.size;
21         arr = new int[size];
22         for (int i = 0; i < size; i++) arr[i] = other.arr[i];
23         return *this;
24     }
```

## Practice 7: spot the self-assignment bug

Find the bug.

```
1 List& operator=(const List& other) {  
2     if (this == *other) return *this;  
3     delete[] arr;  
4     size = other.size;  
5     arr = new int[size];  
6     for (int i = 0; i < size; i++) arr[i] = other.arr[i];  
7     return *this;  
8 }
```

## Practice 7 Answer

The self-assignment check is wrong: it compares a pointer to an object value.

Correct check:

```
1 | if (this == &other) return *this;
```

# Rule of 5

- 1 Memory Regions
- 2 Stack
- 3 Heap
- 4 Pointers
- 5 Dynamic 1D Arrays
- 6 2D Arrays
- 7 References vs Pointers
- 8 Classes Owning Dynamic Memory
- 9 Rule of 3
- 10 Rule of 5**
- 11 Tools
- 12 Review

# Why add move operations

Rule of 3 is correct, but copies can be expensive.

- Copy duplicates the entire dynamic array
- Temporary objects and returns-by-value can cause copies
- Move semantics allow stealing the resource pointer



# Rule of 5

Rule of 5 adds:

- Move constructor
- Move assignment operator

Idea:

- Steal pointer from a temporary
- Set the source pointer to nullptr so its destructor is safe

# Move constructor example

```
1 List(List&& other) {  
2     size = other.size;  
3     arr = other.arr;  
4  
5     other.size = 0;  
6     other.arr = nullptr;  
7 }
```

# Move assignment example

```
1 List& operator=(List&& other) {  
2     if (this == &other) return *this;  
3  
4     delete[] arr;  
5  
6     size = other.size;  
7     arr = other.arr;  
8  
9     other.size = 0;  
10    other.arr = nullptr;  
11  
12    return *this;  
13 }
```

## Practice 8: what must be true after a move

After moving from an object, what properties should the moved-from object have?

- It should be safe to destroy
- It should not own the resource anymore
- It should be in a valid state for assignment

## Practice 8 Answer

A common safe moved-from state:

- pointer is nullptr
- size becomes 0
- destructor does nothing harmful
- assignment can later give it a new resource

# Tools

- 1 Memory Regions
- 2 Stack
- 3 Heap
- 4 Pointers
- 5 Dynamic 1D Arrays
- 6 2D Arrays
- 7 References vs Pointers
- 8 Classes Owning Dynamic Memory
- 9 Rule of 3
- 10 Rule of 5
- 11 Tools**
- 12 Review

# valgrind on Linux

valgrind can detect most memory management problems.

- Memory leaks
- Use-after-free
- Invalid reads and writes

Note: not natively available on macOS in many setups.

# valgrind commands

```
1 | valgrind --tool=memcheck --leak-check=yes ./main
2 | valgrind --tool=memcheck --leak-check=yes --show-leak-kinds=all
   | ↪ ./main
3 | valgrind --tool=memcheck --leak-check=yes --show-leak-kinds=all
   | ↪ --track-origins=yes ./main
```



# Review

- 1 Memory Regions
- 2 Stack
- 3 Heap
- 4 Pointers
- 5 Dynamic 1D Arrays
- 6 2D Arrays
- 7 References vs Pointers
- 8 Classes Owning Dynamic Memory
- 9 Rule of 3
- 10 Rule of 5
- 11 Tools
- 12 Review**

# Summary

- Stack memory is automatic and tied to function calls
- Heap memory is manual: `new` allocates and `delete` frees
- Pointers store addresses; dereference only when valid
- `new` matches `delete`, `new[]` matches `delete[]`
- Dynamic arrays need correct cleanup patterns
- Resource-owning classes must avoid shallow copy
- Rule of 3 makes copies safe; rule of 5 makes temporaries fast
- Tools like `valgrind` help locate memory errors

# Review questions

- What lives on the stack and what lives on the heap
- What does `nullptr` represent, and why is it useful
- Why must `new[]` be paired with `delete[]`
- What is the shallow copy problem
- What are the three members in rule of 3
- What is the main idea of move semantics

# Review question answers

- Stack: locals and parameters. Heap: dynamically allocated objects and arrays.
- `nullptr` means no object; it helps represent optional pointers safely.
- `new[]` allocates an array; `delete[]` destroys the array correctly.
- Shallow copy copies pointer values, making multiple objects share one heap block.
- Rule of 3: destructor, copy constructor, copy assignment operator.
- Move semantics steals the resource pointer from a temporary and nulls out the source.